MAE 6291
Internet of Things for Engineers

Prof. Kartik Bulusu, MAE Dept.

Week 3 [02/04/2026]	• Setting up the Edge Lab • IoT Architecture and Ecosystem • Layers in IoT systems - 3 level layer model	• The Linux Terminal • Blinking LEDs on using MQTT [Graded lab activity]

git clone https://github.com/gwu-mae6291-iot/spring2026_codes.git

School of Engineering & Applied Science
THE GEORGE WASHINGTON UNIVERSITY

Spring 2026

Photo: Kartik Bulusu

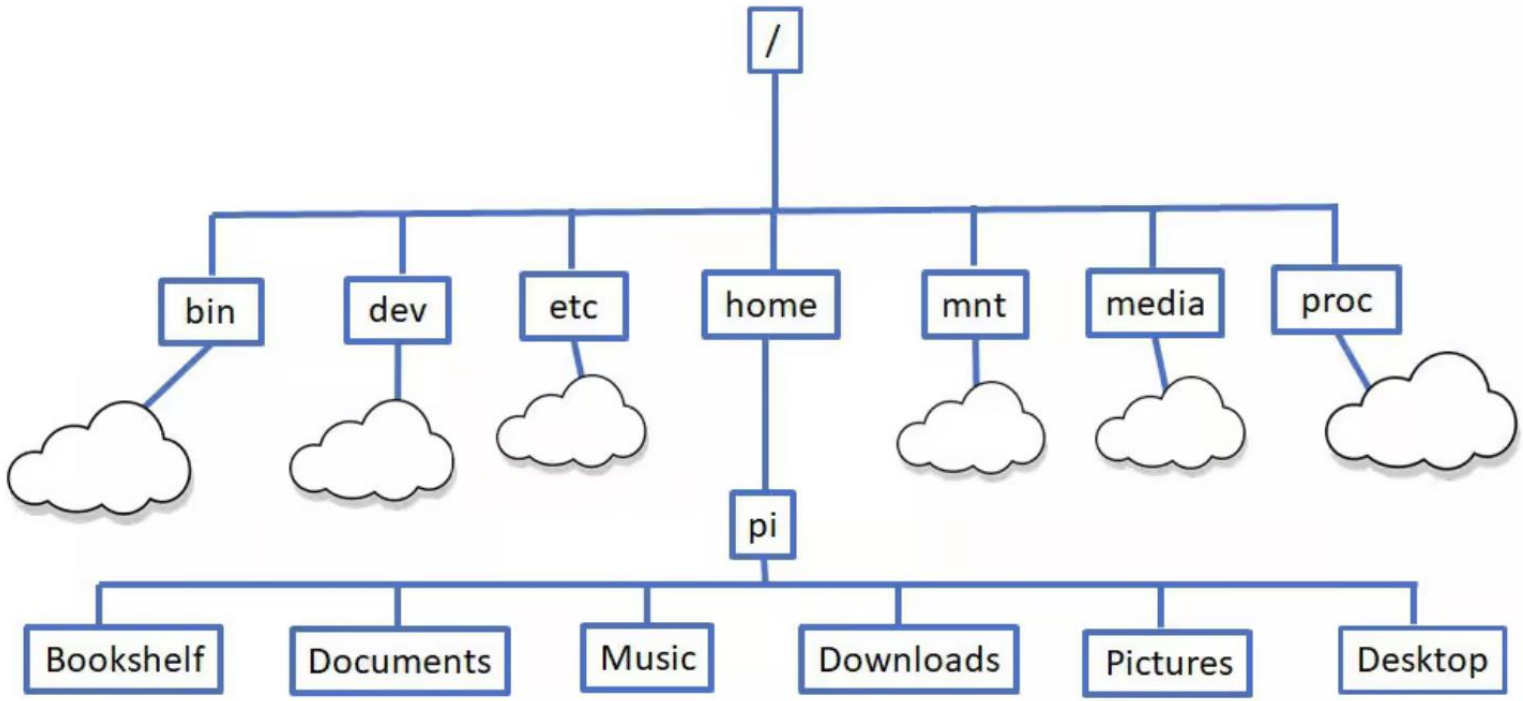
Content Attribution & Sources

This course material incorporates both original content and supplementary visual aids:

- **Images with references:** Properly cited in slide captions, figure legends, or in-text attributions
- **Unreferenced images:** Sourced from Perplexity.io and used for illustrative purposes in an educational context
- **Fair use:** All materials are used in compliance with educational fair use guidelines

The Linux Terminal

Source:
https://www.circuitbasics.com/how-to-navigate-the-raspberry-pi-file-system/



Explore files and folders

Command	Description
pwd	Show the **current** directory you are in
ls	List files in the current directory; add -l or -a for more detail or to see hidden files
cd /path/goes/here	Change directory (for example cd /home/pi/Documents)
mkdir myfolder	Create a new directory named myfolder
cp file1 file2	Copy a file; cp -r dir1 dir2 copies a whole directory
mv old new	Move or rename files/directories
rm file	Delete a file; use rm -r folder carefully to remove a directory

Helpful navigation and history tips

Command or Keyboard Entry	Description
clear	Clear the terminal screen.
history	Show a numbered list of past commands; run one again with !number.
Up/Down arrow keys	Scroll through previously used commands to run or edit them.

View and edit files

Command	Description
nano file.txt	Open a simple text editor in the terminal to create or edit a file
less file.txt	View a long file one page at a time (use q to quit)
cat file.txt	Print the contents of a text file to the screen

System information and status

Command	Description
hostname -I	Show the Pi's IP address
uname -a	Show Linux kernel and system details
df -h	Show disk space usage in human-readable form
free -h	Show memory (RAM) usage
uptime	See how long the Pi has been running
top or htop	Monitor running processes and CPU usage (press q to exit top).

IP address of your RPi connected to the network using WiFi

ifconfig -a

```
pi@raspberrypi017: ~
File  Edit  Tabs  Help
pi@raspberrypi017:~ $ ifconfig -a
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether ea:e0:b9:19:85:8f  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 169.254.247.201  netmask 255.255.0.0  broadcast 169.254.255.255
        inet6 fe80::21e4:d41c:791a:a9d7  prefixlen 64  scopeid 0x20<link>
        ether dc:a6:32:98:38:24  txqueuelen 1000  (Ethernet)
        RX packets 1874  bytes 166519 (162.6 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1723  bytes 1258105 (1.1 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 53  bytes 5297 (5.1 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 53  bytes 5297 (5.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.198.24.23  netmask 255.255.252.0  broadcast 10.198.27.255
        inet6 fe80::3d8a:ccb:6a3a:2f0  prefixlen 64  scopeid 0x20<link>
        ether dc:a6:32:98:38:25  txqueuelen 1000  (Ethernet)
        RX packets 12  bytes 1379 (1.3 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 67  bytes 8066 (7.8 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

pi@raspberrypi017:~ $
```

Take a tiny quiz on Linux terminal and feel good!

Use the following link:

https://tinyurl.com/7aj4m2c8

OR

Some rules to observe:

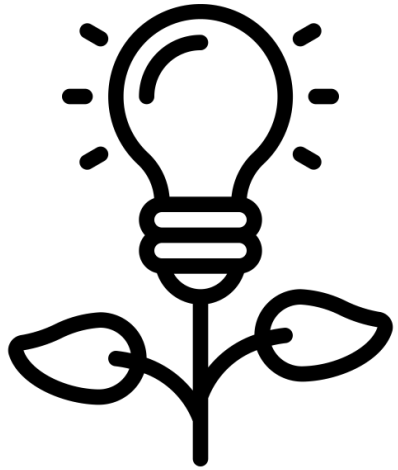
- 10 minutes restriction
- Unlimited responses
- You can discuss with your colleagues in-class
- Watch real-time results and calibrate

Icon source:
https://thenounproject.com/icon/clean-energy-4339399/

Two questions up for discussion

1. How should we start perceiving an IoT system, physically ?

2. How / Where do we place the “thing” in that system ?



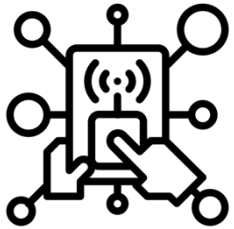
Keywords:
Small, functional, re-envisioning applications, efficient, sensor-driven, smart-sensors, connectivity, autonomy, data-driven, durability, fault tolerance, interoperability

Proximity to data, compute-power, network, distributed-network etc.

Building up the IoT Architecture and Ecosystem

Icon Sources:
sensor by Carolina Cani:, sensor by Pham Duy Phuong Hung, sensor by Tippawan Sookruay, sensor by Lorenzo: https://thenounproject.com/browse/icons/term/sensor
fire sensor by LAFS : https://thenounproject.com/browse/icons/term/fire-sensor/
Ultrasound by Shocho: https://thenounproject.com/browse/icons/term/ultrasound/
Network by Solikin:, Network by Tippawan: https://thenounproject.com/browse/icons/term/network
application by Chaowalit Koetchuea: https://thenounproject.com/browse/icons/term/application/

Information-layer



Data

Communication-layer

Connectivity

Sensor-layer

Things

Explore MQTT Basics
Message Queuing Telemetry Transport

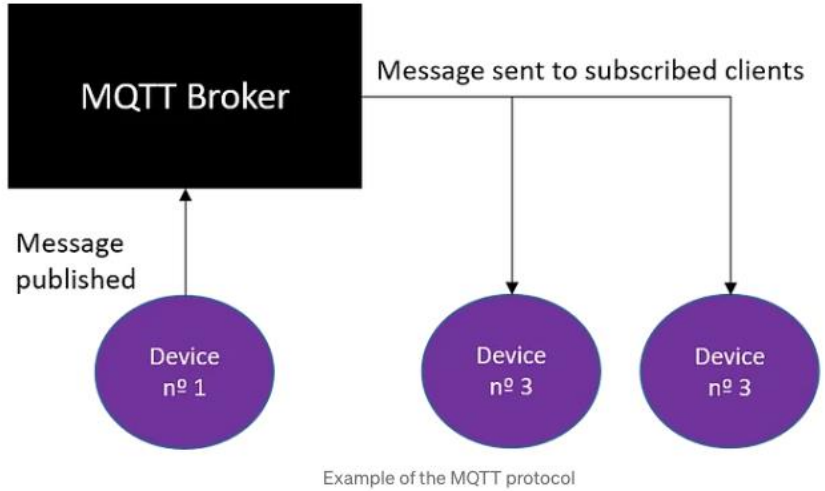
Goal: To understand how publishing and subscribing works practically

School of Engineering & Applied Science
THE GEORGE WASHINGTON UNIVERSITY

GW

Prof. Kartik Bulusu, MAE Dept.

MQTT paradigm



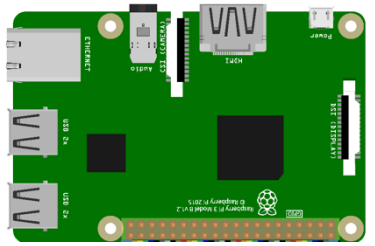
Example of the MQTT protocol

Source:
https://andre-benevides.medium.com/introduction-to-mqtt-and-configuration-of-a-mosquitto-broker-f0f7a7738bc8
https://learn.sparkfun.com/tutorials/introduction-to-mqtt#the-basics

Hardware

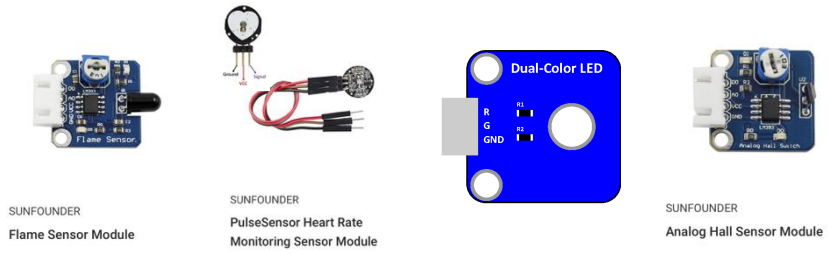
Broker

- The broker is the server
- It distributes the information to the interested devices connected to the server.



Client

- The device that connects to broker to send or receive information.



Messaging

Topic

- The name that the message is about.
- Clients publish, subscribe, or do both to a topic.

Subscribe

- Clients tell the broker which topic(s) they're interested in.

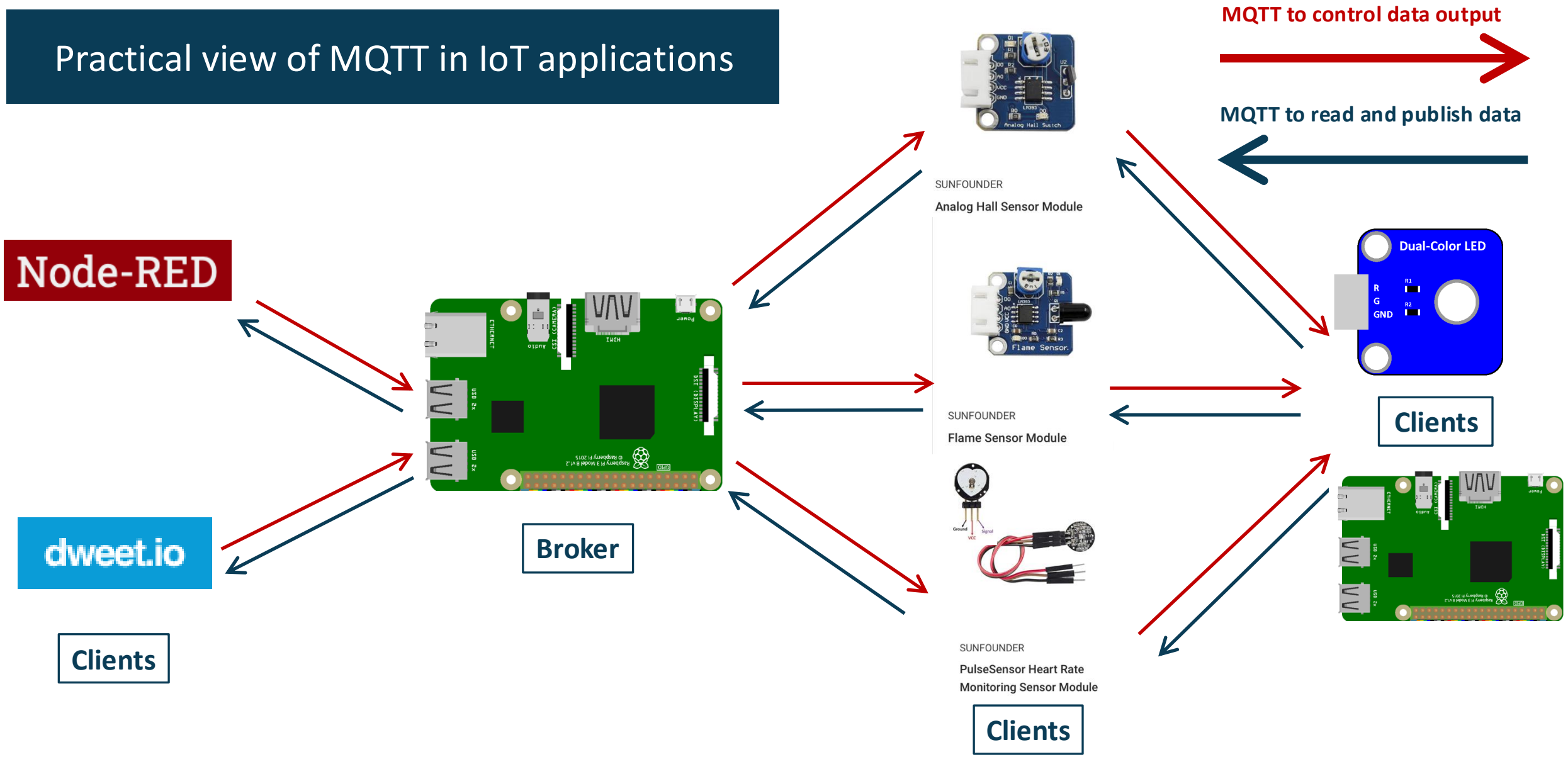
Publish

- Clients that send information to the broker to distribute to interested clients based on the topic name.

QoS

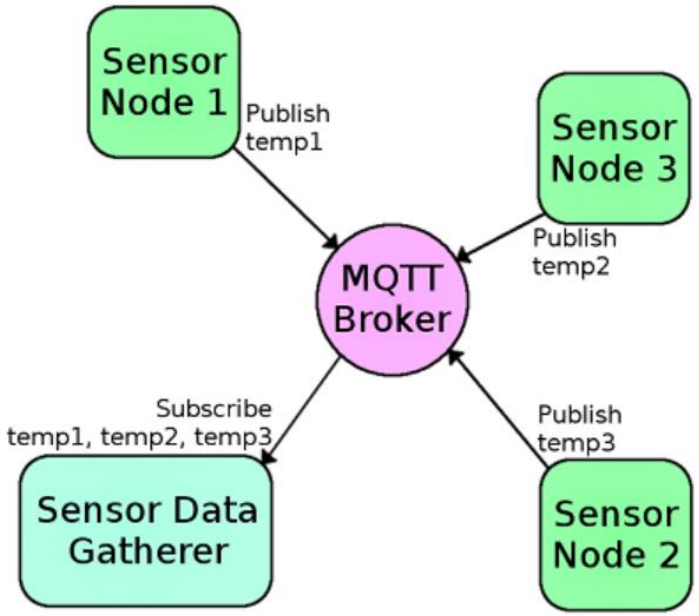
- Quality of Service to the broker
- Integer value ranging from. 0-2.

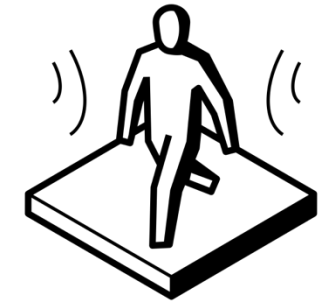
Practical view of MQTT in IoT applications



MQTT paradigm

Source:
https://learn.sparkfun.com/tutorials/introduction-to-mqtt#the-basics
https://andre-benevides.medium.com/introduction-to-mqtt-and-configuration-of-a-mosquitto-broker-f0f7a7738bc8
Health monitoring by Sinta Maulana from Noun Project (CC BY 3.0)
motion sensor by Tomas Knopp from Noun Project (CC BY 3.0)
fire sensor by Carolina Cani from Noun Project (CC BY 3.0)
chat by Rolas Design from Noun Project (CC BY 3.0)

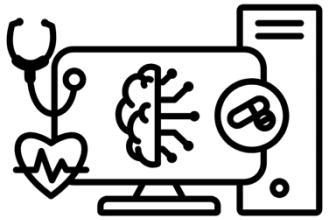




Created by Tomas Knopp from Noun Project

Created by Carolina Cani from Noun Project

Common usages



Created by Sinta Maulana from Noun Project

Created by Rolas Design from Noun Project

Source
https://mqtt.org/getting-started/ :
https://mosquitto.org
https://github.com/eclipse/mosquitto
https://projects.eclipse.org/projects/iot.mosquitto

Eclipse Mosquitto - An open source MQTT broker

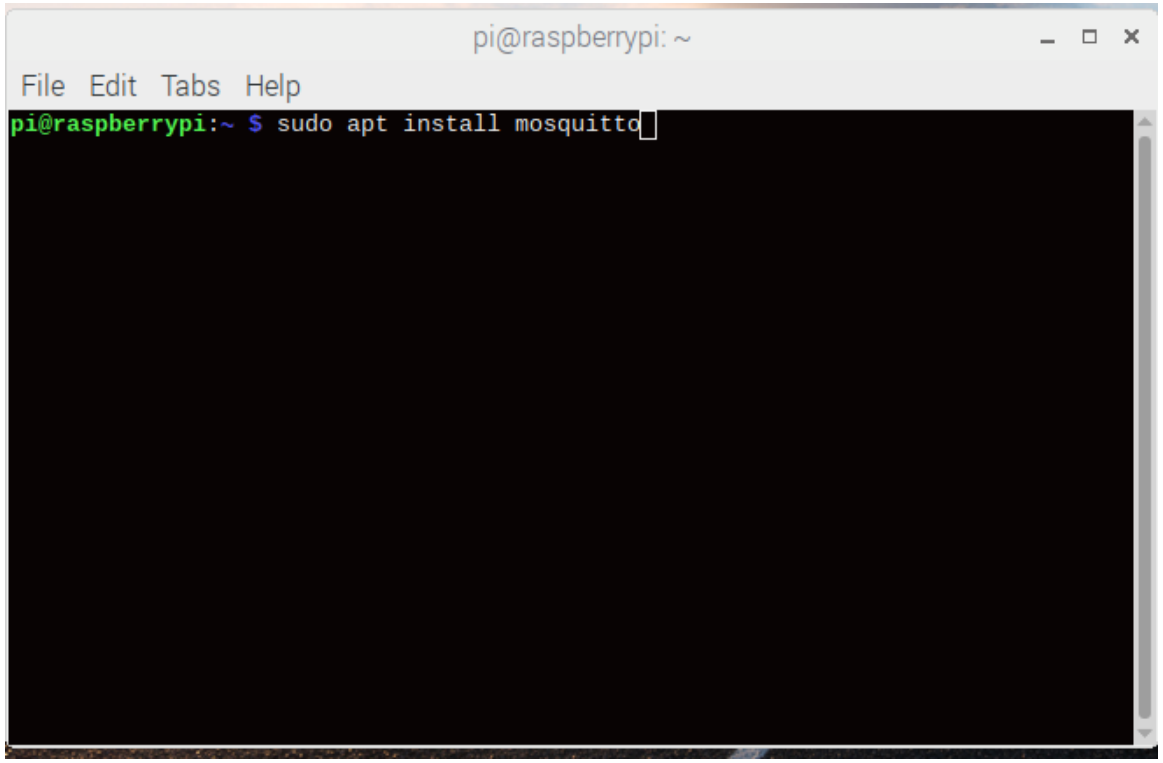
Eclipse Mosquitto provides a lightweight server implementation of the MQTT protocol that is suitable for all situations from full power machines to embedded and low power machines.

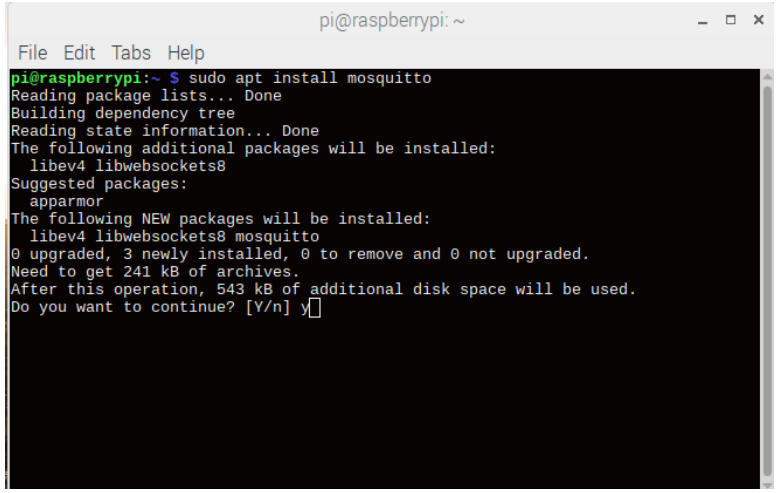
Sensors and actuators, which are often the sources and destinations of MQTT messages, can be very small and lacking in power. This also applies to the embedded machines to which they are connected, which is where Mosquitto could be run.

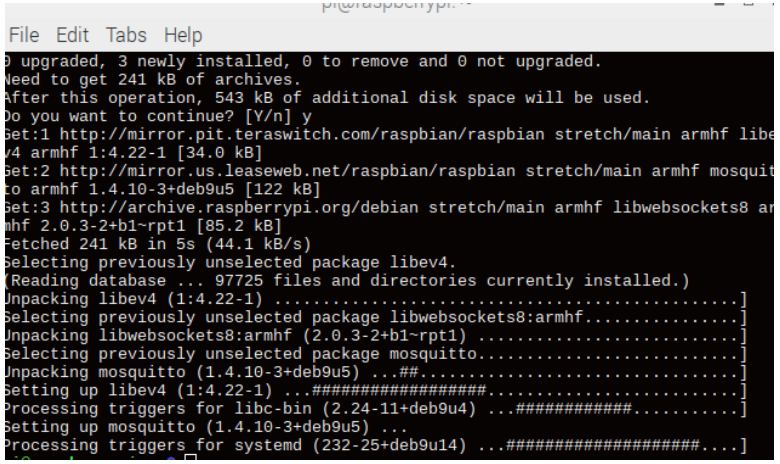
Step-1: Eclipse Mosquitto - An open source MQTT broker

sudo apt-get update
sudo apt-get upgrade

sudo apt install mosquitto

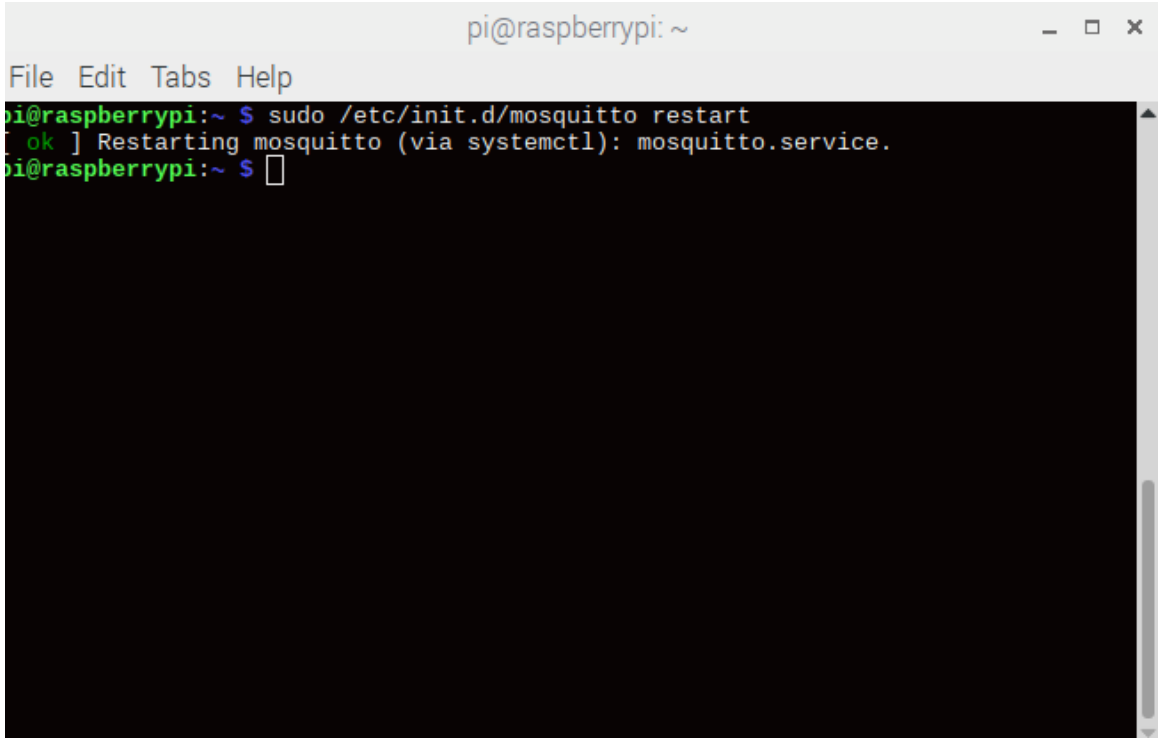






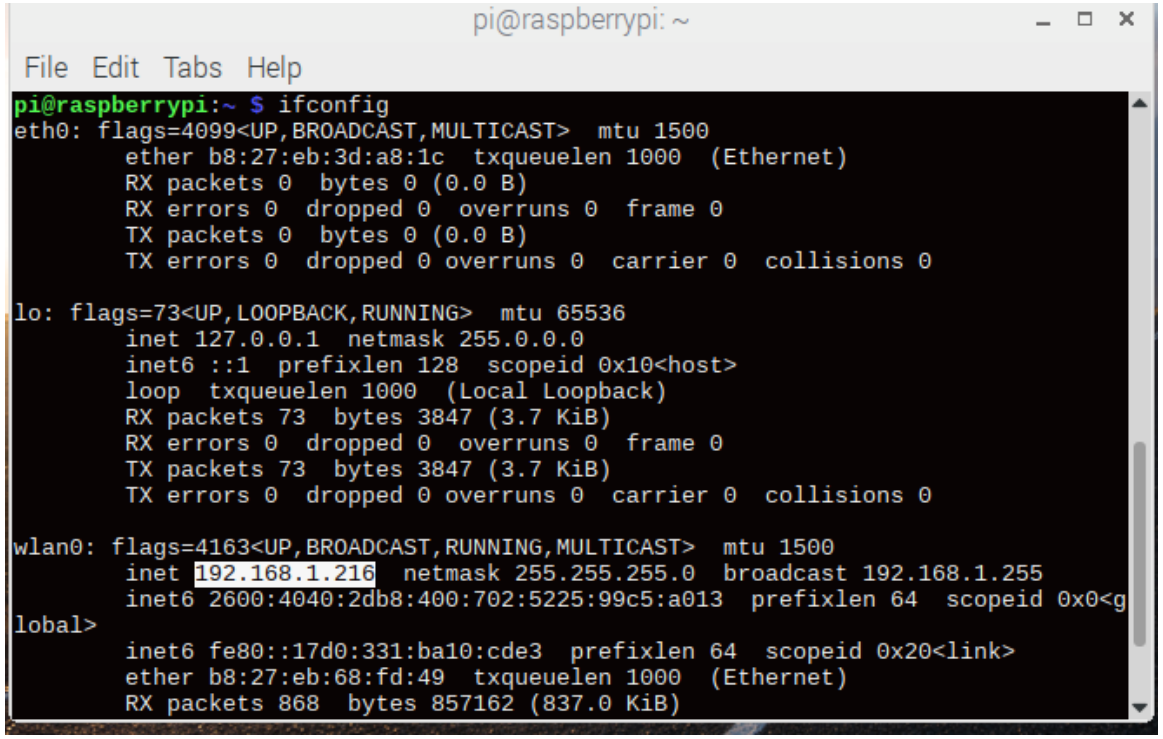
Step-2: restart Mosquitto

sudo /etc/init.d/mosquitto restart



Step-3: Get your IP address

ifconfig

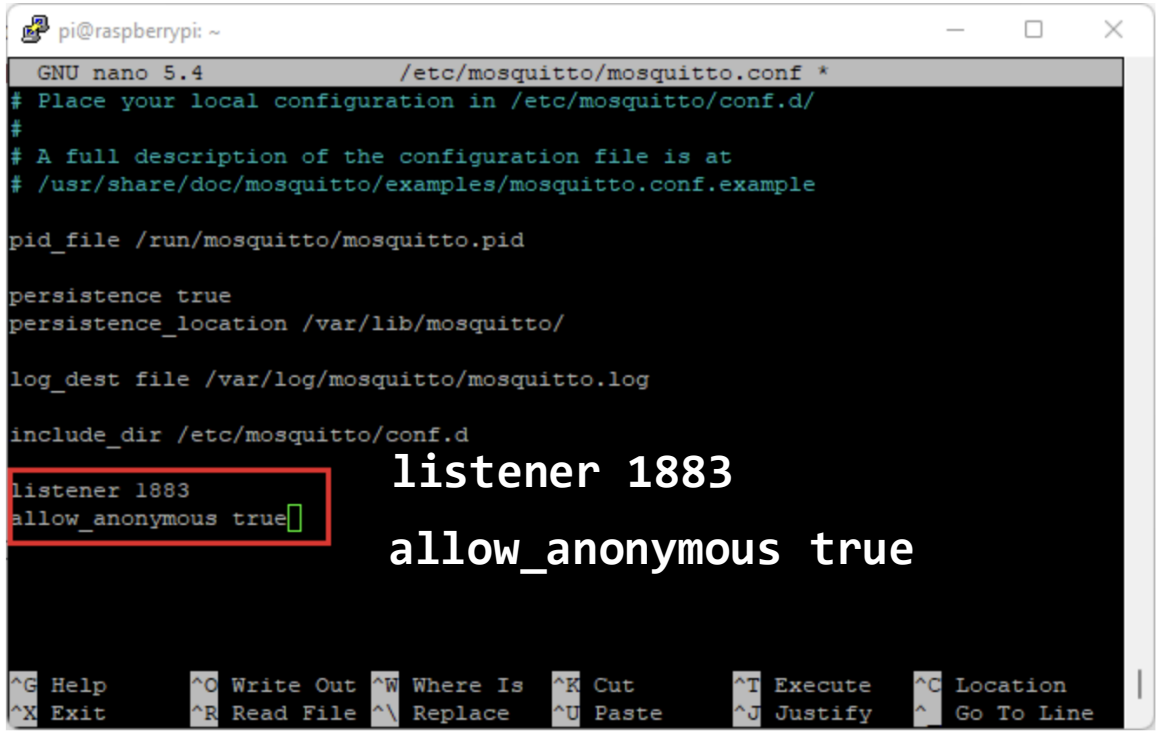


hostname –I # this is also OK to use

Source:
https://randomnerdtutorials.com/how-to-install-mosquitto-broker-on-raspberry-pi/
https://forums.raspberrypi.com/viewtopic.php?t=331096

Step-4: Enable Remote Access to Mosquitto Broker (No Authentication)

sudo nano /etc/mosquitto/mosquitto.conf



sudo systemctl restart mosquitto

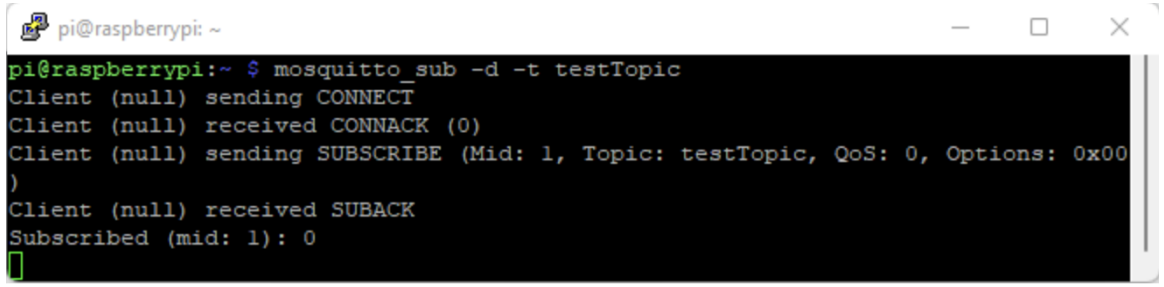
sudo systemctl status mosquitto

Source:
https://randomnerdtutorials.com/testing-mosquitto-broker-and-client-on-raspbbery-pi/
https://mosquitto.org/man/mosquitto_pub-1.html
https://mosquitto.org/man/mosquitto_sub-1.html

Step-4: Publishing "Hello World!" Message to *testTopic* Topic

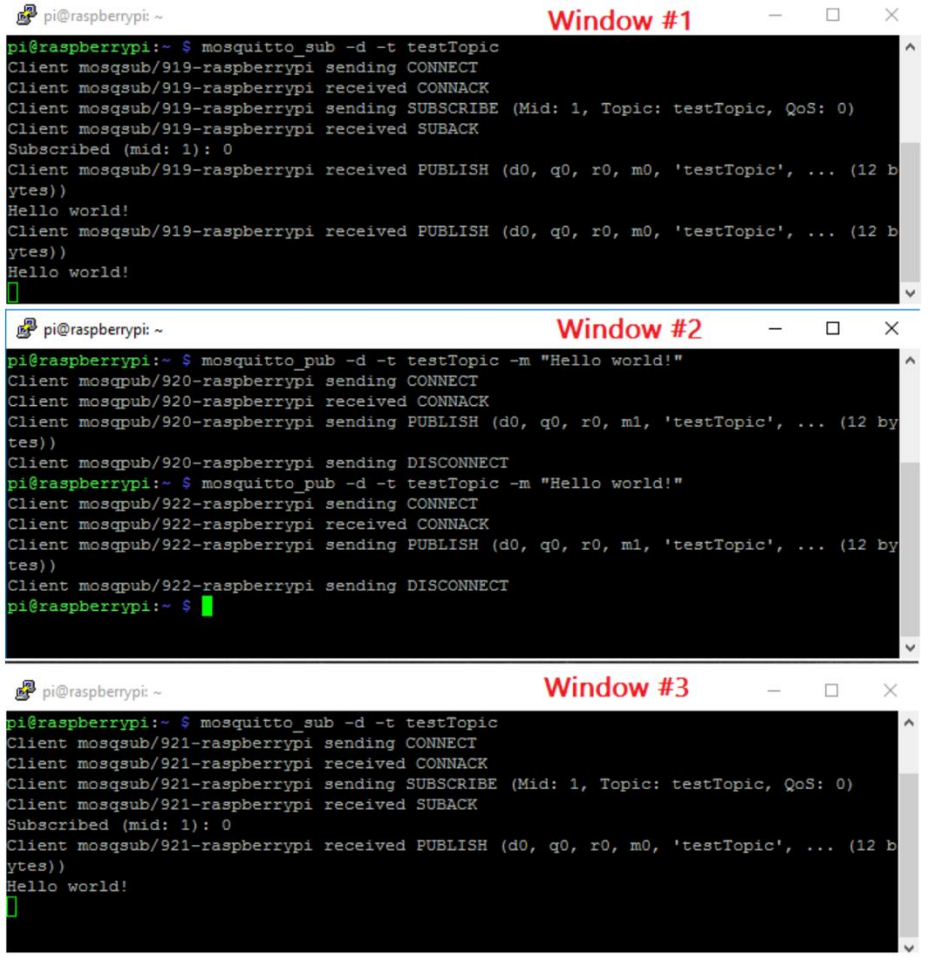
 # Install mosquitto-clients
sudo apt install -y mosquitto mosquitto-clients

 # Open a terminal window and type the following:
mosquitto_sub -d -t testTopic



mosquitto_sub -v -t '#' -h <IP address>

mosquitto_pub -d -t testTopic -m "Hello world!"



Source:
https://andre-benevides.medium.com/introduction-to-mqtt-and-configuration-of-a-mosquitto-broker-f0f7a7738bc8

Types of MQTT messages

CONNECT — Is the client request to connect to the broker

CONNACK — Acknowledgement of the connect

PUBLISH — Publishes a message to a topic

PUBACK — Acknowledgement of the publish with QoS level 1

PUBREC — Acknowledgement of the publish with QoS level 2 (2nd packet)

PUBREL — Response to the PUBREC. (3rd packet when using QoS level 2)

PUBCOMP — Response to PUBREL (4th and last packet when using QoS lvl 2)

SUBSCRIBE — Packet from the client to subscribe to topics

SUBACK — Acknowledgement of the subscribe packet

UNSUBSCRIBE — Packet from the client to unsubscribe from topics

All activities today are a part of graded in-class lab
Download codes from github and demonstrate
[10 points]

Skeleton of the updated Python program written for Raspberry Pi

```
import library1 as name1
import RPi.GPIO as GPIO
import time
```

Import libraries that are relevant for interaction with the Raspberry Pi hardware such as GPIO pins, camera ports etc.

```
GPIO.setmode(GPIO.BOARD)

GPIO.setup(12, GPIO.OUT)
```

Set up GPIO pins as data outputs or inputs for sensors and actuators

```
def function_name(arguments):
        statement
        statement
        . . .
        return value
```

Create user-defined functions to modularize your code and make it easy to work.

Create user-defined functions to release the GPIO pins

```
if __name__ == "__main__":
        try:
                function_name1()
        except KeyboardInterrupt:
                function_name2()
```

Create entry point into the program and pull all functions in

Create a keyboard-interrupt exception clause

Light up an LED with your Raspberry Pi 4 B

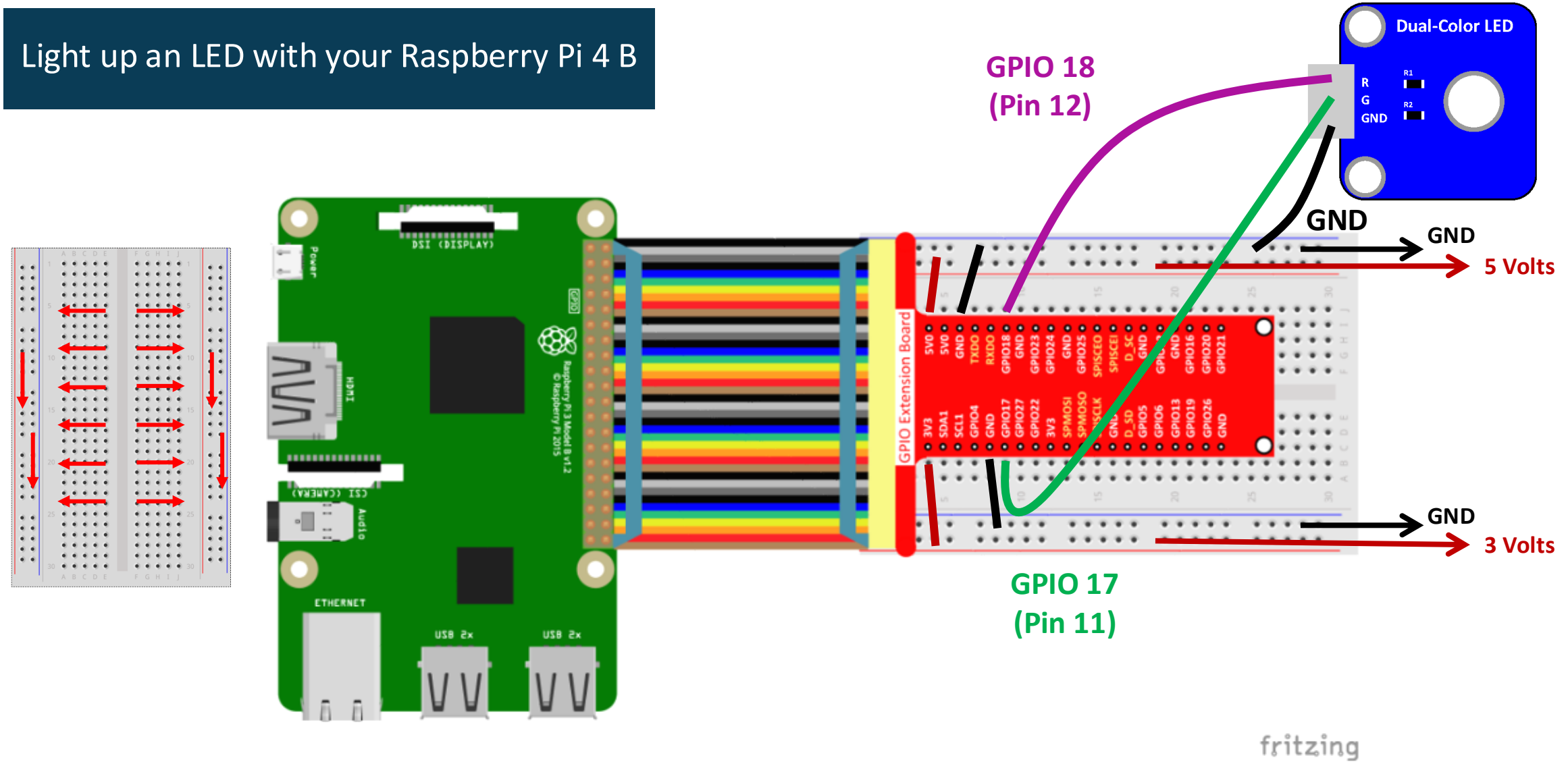



Photo: Kartik Bulusu
Someone should summarize what we learned today